Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs

DAY 5

Publish

Deploy, verify, document, and demonstrate

Thursday, October 29, 2026

Your five-day journey

Yesterday: secured and configurable. Today: live, documented, demonstrated.

01

Foundation

Project, Task model, migrations, admin

DONE

02

Core API

CRUD with function views

DONE

03

Serialization & validation

Serializers, rules, clear field errors

DONE

04

Security & configuration

Tokens, ownership, environment

DONE

05

Publish

Deploy, verify, README, live demo

TODAY

Today's session

Guided deployment replaces the normal exercise hour

Explain + live deploy
~60 min

Guided deployment
~60 min

Break
short

Verify + submit + demo
~60 min

Exit check
end of day

No exercise hour today. The production-readiness exercise is optional homework.

Learning outcomes

By the end of today you can...

01 Release the API to a public HTTPS environment

02 Use Neon PostgreSQL through DATABASE_URL

03 Verify auth, CRUD, validation, ownership, and persistence live

04 Read deployment logs and fix common configuration failures

05 Document, submit, and demonstrate the completed system

Same code, different configuration

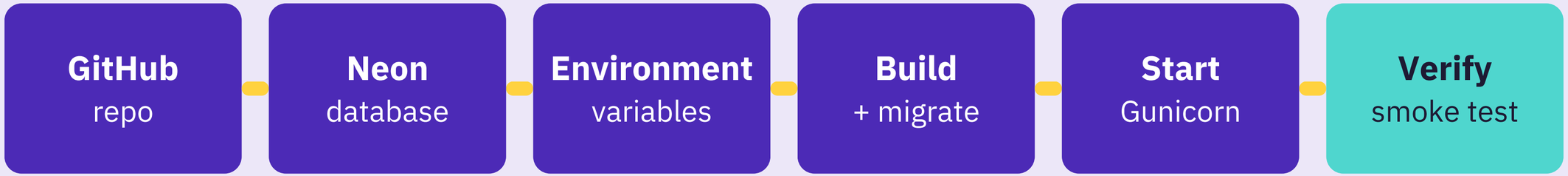
Production changes the runtime environment, not the application's purpose

Setting	Laptop	Render + Neon
Server	runserver	gunicorn
Database	SQLite (db.sqlite3)	PostgreSQL via DATABASE_URL
DEBUG	True	False
Secret key	development key	long random secret
Static files	served by runserver	collected, served by WhiteNoise

Render disk is temporary. Persistent data belongs in PostgreSQL, not in files.

The release sequence

A deploy connects code, infrastructure, configuration, and verification



Deploy first, then prove the public HTTPS URL behaves correctly.

What production needs

Render receives values and commands your laptop supplied automatically

Environment

```
SECRET_KEY
DEBUG=False
ALLOWED_HOSTS
DATABASE_URL
CSRF_TRUSTED_ORIGINS
```

Build

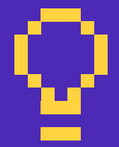
```
pip install -r
requirements.txt
python manage.py
collectstatic --noinput
python manage.py migrate
```

Start

```
gunicorn
config.wsgi:application
```

Public HTTPS URL for production users

Local users are not in the production database. Create production users after deploying.



LIVE DEMO

Reference deploy

The instructor deploys first, start to finish, on Render + Neon

Neon
database

Render
web service

Environment
variables

Build
+ start

Smoke test
live URL

Watch the logs as it builds. You will read your own next.

Reading deployment logs

Find the first meaningful error, then classify the failure

Dependencies
version conflict · missing package · requirements.txt

Configuration
SECRET_KEY · ALLOWED_HOSTS · CSRF origin · DEBUG

Database + startup
connection string · migrations · Gunicorn module

Read the first error that explains why the build or service stopped. Later errors are often consequences.

What done looks like today

A preview of hours 2 and 3

01 Smoke test passes 10/10 against your live URL

02 A task survives a restart or redeploy

03 The README lets a reviewer set up from a fresh clone

04 Your submission is in before your demo

05 The five-minute live demo passes

HOUR 2 · GUIDED DEPLOYMENT

Everyone deploys together

Follow the instructor step by step. Nobody deploys alone.

Prepare the code → Neon database → Render service → Production users

Prepare and deploy

The checkpoint keeps your work and adds the production files

On track

Copy only config/settings.py, .env.example, and smoke_test.py. Complete P1 DATABASE_URL and P2 .env.example.

Local check

`python manage.py check`
`python manage.py collectstatic --noinput`
then commit and push

Infrastructure

Neon database, Render web service, environment variables, build and start

Behind? Copy the whole Day 5 checkpoint, install requirements, and run migrations.

BREAK

Short break

After the break: verify, document, submit, and demonstrate

HOUR 3 · VERIFY AND DEMO

Prove it live

A deploy becomes a finished project only after production behavior is proven

Public HTTPS URL

Production users

Smoke test 10/10

Restart service

Data still there

Verify the live service

Run smoke_test.py against the real onrender.com URL; expect 10/10 checks passed

Operation	Live API path	Expect
Create	`POST /api/tasks/create/`	201
Read	`GET /api/tasks/{id}/`	200
Update	`PUT / PATCH /api/tasks/{id}/update/`	200
Delete	`DELETE /api/tasks/{id}/delete/`	204

Check authentication, validation and two-user isolation.
Create a task; restart or redeploy; list it again to prove persistence.
Save the 10/10 output without tokens or passwords.

Free Render services sleep after about 15 idle minutes. Open your URL before the demo.

README and submission evidence

Documentation and smoke-test evidence are part of the deliverable

README · P3

Setup, environment, migrations, authentication, endpoints, live URL, deployment

Submit

GitHub URL, public HTTPS URL, smoke-test output (10/10), Definition of Done

Safe evidence

No tokens, passwords or database URLs. Repository accessible to reviewers.

Submit as soon as the smoke test passes, before your live demo.

The five-minute live demonstration

Use your live URL at one of 2–3 parallel review stations

01 Authenticate
obtain a token

02 Create + list
show the task

03 Update + fail
show a 400

04 Isolation
second user gets 404

05 Delete + persist
restart, data remains

For every step: name the claim, make the request, point at the status code and body.

Day 5 exit check

The program is complete when every check passes

01 Public HTTPS URL is reachable; production migrations applied

02 Token authentication and all CRUD operations work in production

03 Isolation and validation pass; a created record survives restart

04 The README enables setup from a fresh clone

05 10/10 smoke output and checklist submitted; live demo passes

Optional improvements begin only after every required item passes

Optional homework: production readiness

Only after your project is deployed and demonstrated

Task 1 · DATABASE_URL

Switch between SQLite and another database URL

Task 2 · DEBUG=False

WhiteNoise, collectstatic, and a styled admin

Task 3 · Smoke test

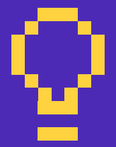
7 live HTTP checks;
7/7 checks passed

The exercise smoke test sends real requests to a running server.
Production data belongs in PostgreSQL because Render disk is temporary.

Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs



Program complete

A public, documented Task Management API.

You can now design, build, document, and deploy a REST API